

DATA-260 Homework 3 – Report

Uday Patel

Section 0 – Personal Configuration and Domain

Value	Result
SID4	9871
PORT_BASE	8871
PREFIX	s9871
SEED	9871
VERIFY_SEED	269871
DOMAIN_ID	7 – Community sports league fixtures

Hardware: Lenovo Legion Slim 5 16” (Ryzen 5 7640HS, 16GB RAM, NVIDIA RTX 4060 8GB, 512GB SSD)

Local model used: sentence-transformers/all-MiniLM-L6-v2 (embeddings, Part 2); qwen3:8b via Ollama (agent work carried over from HW1/HW2)

Repository: <https://github.com/UdayPate/data260-9871>

Tagged commit: e9d4e08d9f603f38d94b482d04bb9bdfae64c439
<https://github.com/UdayPate/data260-9871/tree/e9d4e08d9f603f38d94b482d04bb9bdfae64c439>

Part 1 – FastAPI User Authentication

A FastAPI authentication router (`code/web.application/auth.py`) implements Home, Login, Dashboard, and Logout routes for the Community Sports League Fixtures domain, using Starlette’s `SessionMiddleware` for signed session cookies plus a server-side session store (`ACTIVE_SESSIONS`) so that logout and idle timeout are genuinely enforced – not merely by clearing the client-side cookie, which alone cannot prevent replay of an already-issued, still-validly-signed cookie.

Route screenshots

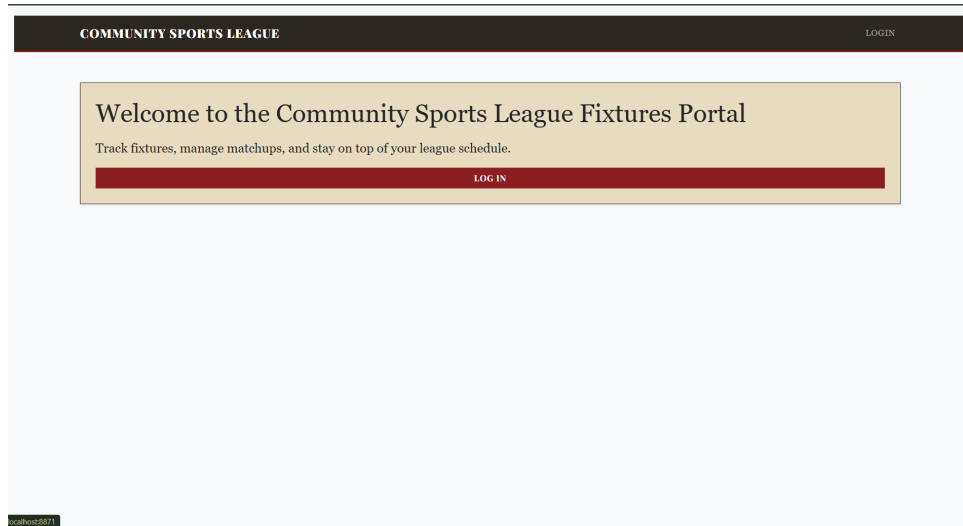


Figure 1: Home page (`/`), logged out – shows only a Login link.

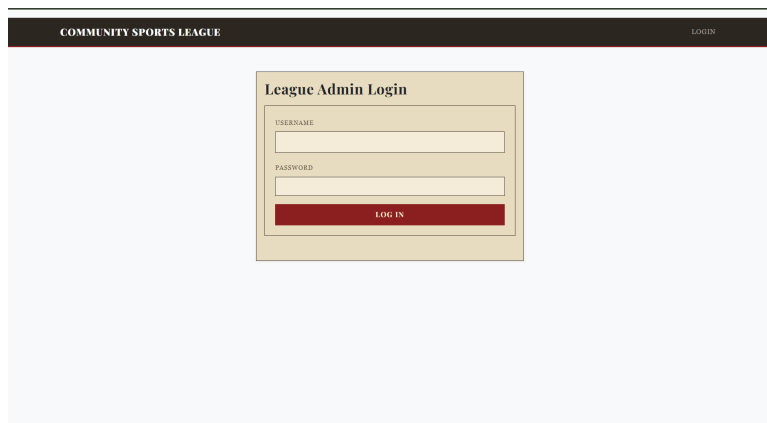


Figure 2: Login page (`/login`), clean state.

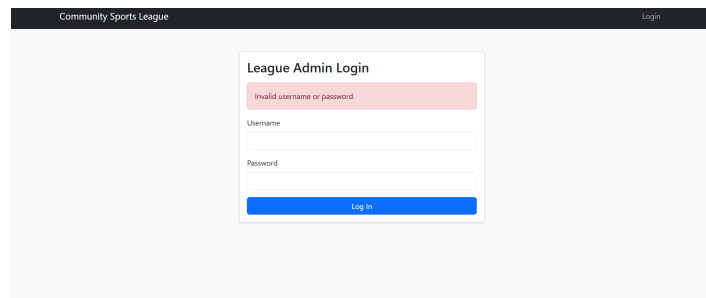


Figure 3: Login page showing the Bootstrap alert on invalid credentials. This screenshot was captured before the vintage theme (Section: Styling) was layered on top of Bootstrap's default styling – the alert/validation behavior was verified working first, then the visual theme was applied afterward and confirmed not to break it (see Figure 2, same alert styling class, after theming).

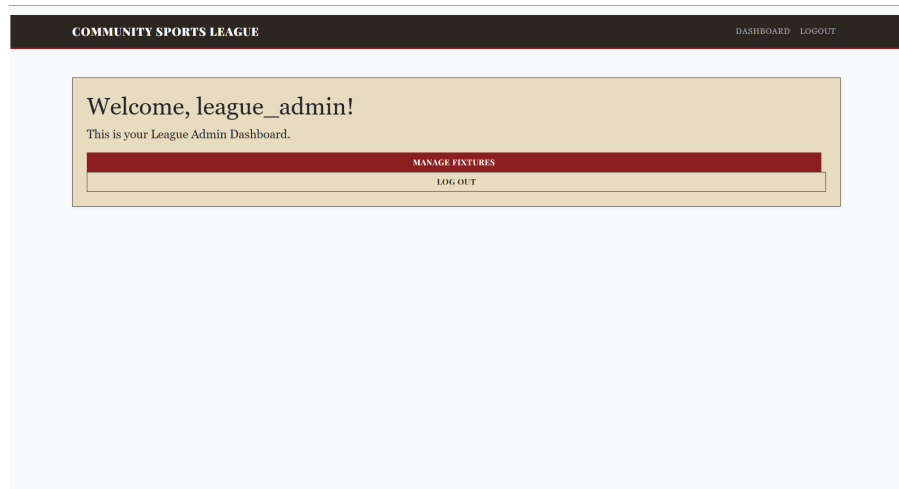


Figure 4: Dashboard page (`/dashboard`), logged in – protected route, shows the logged-in username and links to Manage Fixtures (the HW2 board) and Logout.

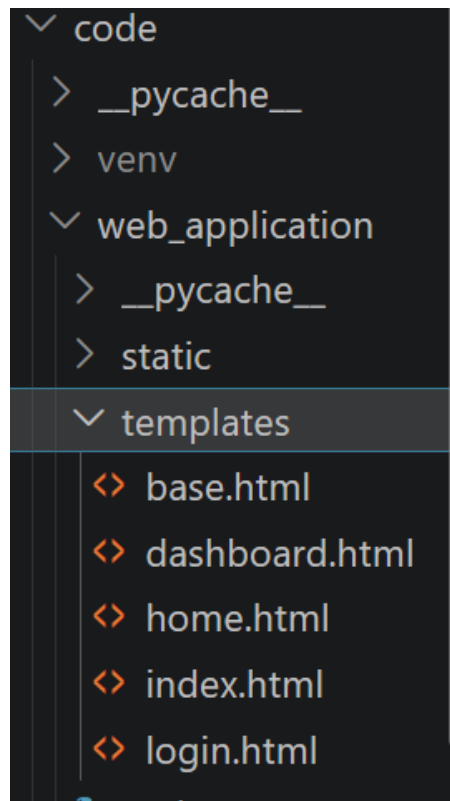


Figure 5: The `templates/` directory – `base.html`, `dashboard.html`, `home.html`, `login.html` (new for HW3) alongside `index.html` (the HW2 fixtures board template).

Session cookie security

The cookie is configured via Starlette's `SessionMiddleware` with `same_site="lax"` and `https_only=True`; `httponly` is enabled by default. The real `Set-Cookie` response header was captured directly from a login request:

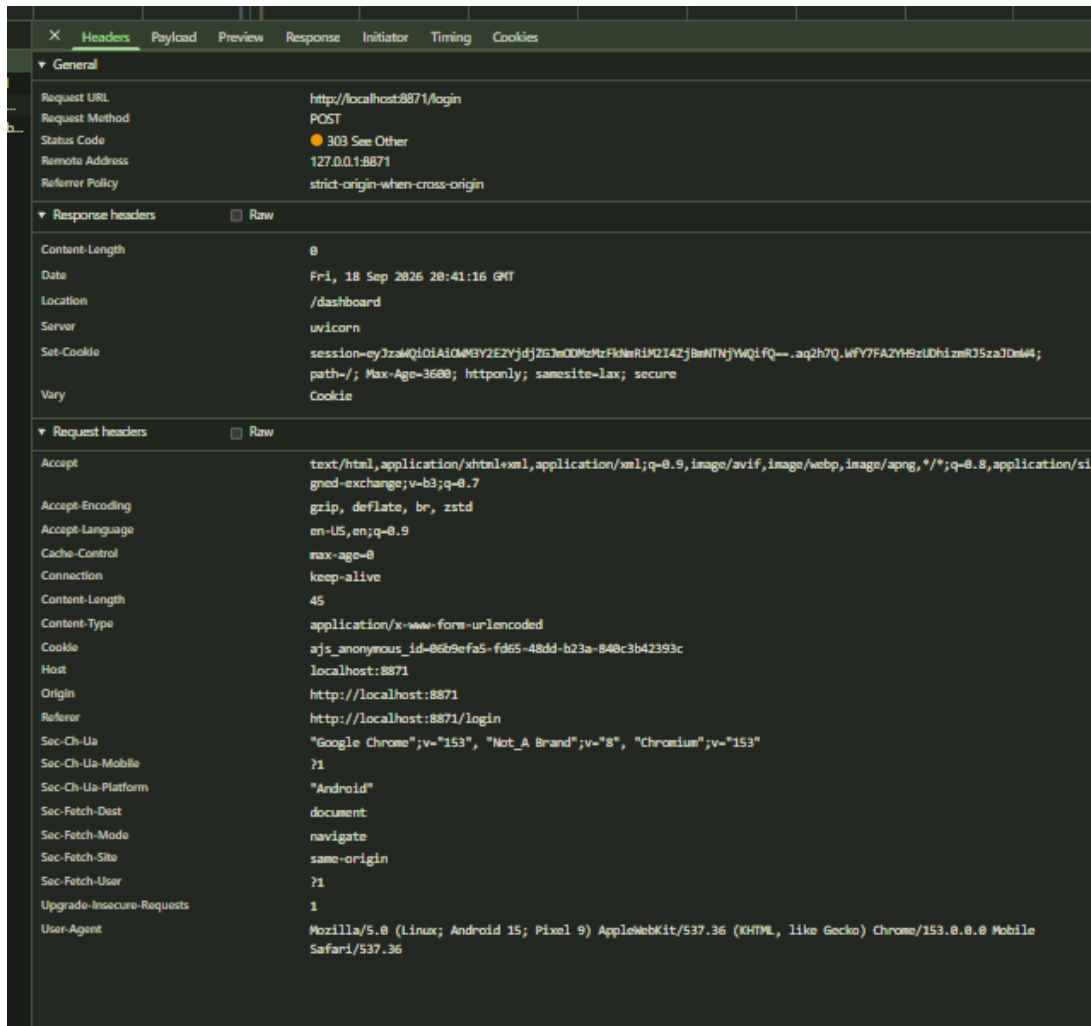


Figure 6: Raw `Set-Cookie` header from a real login request, captured via Chrome DevTools. All three required attributes are visible: `httponly`, `samesite=lax`, `secure`.

Proof: a logged-out or expired session cannot reach /dashboard

Because `SessionMiddleware` cookies are only *signed*, not encrypted or server-tracked, a naive implementation that merely clears the client-side cookie on logout does not actually prevent an already-issued cookie from being replayed – the signature is still valid. `auth.py` closes this gap with a server-side `ACTIVE_SESSIONS` store: logout deletes the session id from that store, and an idle-timeout check does the same after 30 seconds of inactivity. Both were verified for real via `curl`, reusing the exact same cookie value across requests:

```
--- Logout replay test ---
POST /login (correct credentials) -> 303 See Other -> /dashboard
Set-Cookie: session=<value>; path=/; Max-Age=3600; httponly; samesite=lax; secure

GET /dashboard (with the cookie) -> "Welcome, league_admin!"
```

```

GET /logout (with the SAME cookie) -> 303 See Other -> /

REPLAY the exact same (now logged-out) cookie against /dashboard:
GET /dashboard (with the OLD cookie) -> 303 See Other -> /login <-- REJECTED

--- Idle timeout test (independent of logout) ---
POST /login (correct credentials) -> 303 See Other -> /dashboard
GET /dashboard (immediately, with the cookie) -> "Welcome, league_admin!"

... waited 32 seconds with NO activity (idle_timeout = 30s) ...

GET /dashboard (SAME cookie, no logout was ever called) -> 303 See Other -> /login <-- REJECTED

```

In both cases the exact same cookie value was rejected on the second request despite being identical to the one that worked moments earlier – proving the rejection came from server-side state (ACTIVE_SESSIONS), not from anything visible to or controllable by the client.

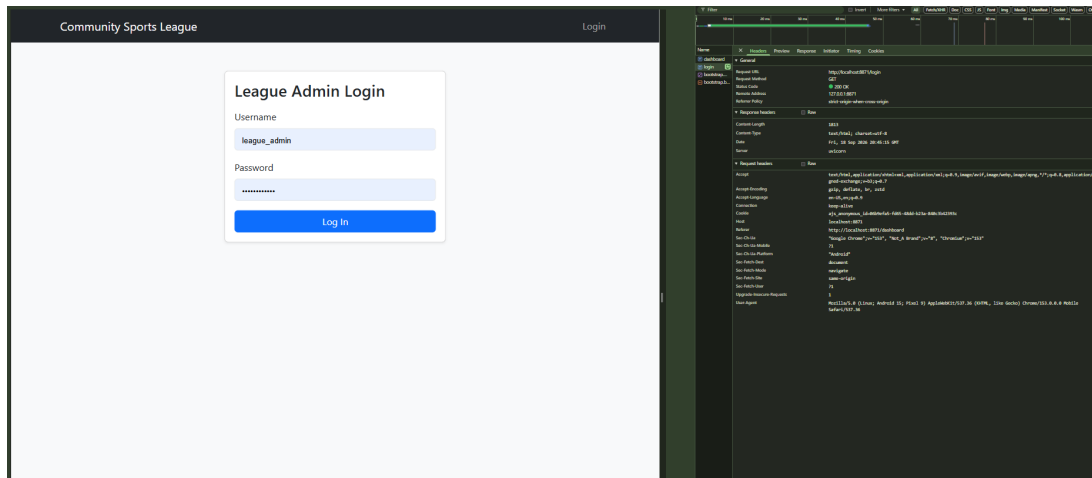


Figure 7: Browser-side view during idle-timeout testing – DevTools Network tab shows the second `/dashboard` request receiving a 303 redirect to `/login` with no intervening logout click.

Full auth.py source

```
"""
DATA-260 Homework 3 - Part 1: Authentication router
Implements Home, Login, Dashboard, and Logout routes for the Community
Sports League Fixtures domain, using Starlette's SessionMiddleware for
session cookies plus a small server-side session store so that logout
and idle timeout can be genuinely enforced (not just clearing the
client-side cookie, which alone cannot prevent replay of an
already-issued cookie).
"""

import secrets
import time
from pathlib import Path

from fastapi import APIRouter, Form, Request
from fastapi.responses import RedirectResponse
from fastapi.templating import Jinja2Templates

router = APIRouter()
templates = Jinja2Templates(directory=Path(__file__).resolve().parent / "templates")

IDLE_TIMEOUT_SECONDS = 30

# Single hardcoded demo account (no user database for this assignment's scope)
VALID_USERNAME = "league_admin"
VALID_PASSWORD = "GoLions2026!"

# Server-side session store: session_id -> {"username": ..., "last_activity": float}
# This is what makes logout and idle-timeout ACTUALLY enforceable - a signed
# session cookie alone can still be replayed after "logout" unless the server
# independently tracks which session ids are currently valid.
ACTIVE_SESSIONS = {}

def get_current_user(request: Request):
    """Returns the logged-in username if the session is valid and has not
    idle-timed-out. Otherwise revokes/clears the session and returns None."""
    sid = request.session.get("sid")
    if not sid or sid not in ACTIVE_SESSIONS:
        request.session.clear()
        return None

    session_data = ACTIVE_SESSIONS[sid]
    if time.time() - session_data["last_activity"] > IDLE_TIMEOUT_SECONDS:
        del ACTIVE_SESSIONS[sid]
        request.session.clear()
        return None

    # Still valid - sliding idle timeout: refresh on activity
    session_data["last_activity"] = time.time()
    return session_data["username"]

@router.get("/")
def home(request: Request):
    username = get_current_user(request)
    return templates.TemplateResponse(request, "home.html", {"username": username})
```

```

@router.get("/login")
def login_form(request: Request):
    return templates.TemplateResponse(request, "login.html", {"error": None, "username": None})

@router.post("/login")
def login_submit(request: Request, username: str = Form(...), password: str = Form(...)):
    if username == VALID_USERNAME and password == VALID_PASSWORD:
        sid = secrets.token_hex(16)
        ACTIVE_SESSIONS[sid] = {"username": username, "last_activity": time.time()}
        request.session["sid"] = sid
        return RedirectResponse(url="/dashboard", status_code=303)

    return templates.TemplateResponse(
        request,
        "login.html",
        {"error": "Invalid username or password.", "username": None},
        status_code=401,
    )

@router.get("/dashboard")
def dashboard(request: Request):
    username = get_current_user(request)
    if not username:
        return RedirectResponse(url="/login", status_code=303)
    return templates.TemplateResponse(request, "dashboard.html", {"username": username})

@router.get("/logout")
def logout(request: Request):
    sid = request.session.get("sid")
    if sid and sid in ACTIVE_SESSIONS:
        del ACTIVE_SESSIONS[sid]
    request.session.clear()
    return RedirectResponse(url="/", status_code=303)

```

Part 2 – Compare Three LlamaIndex Chunking Techniques (Retrieval-Only RAG)

Domain corpus

Six real, public documents were downloaded and verified, covering all four sports in `DOMAIN_SCHEMA.md` plus one general multi-sport policy document, totaling 232,807 bytes (227.4 KB), exceeding the 200KB requirement:

File	Sport	Size
soccer_ayso_basic_rules.txt	Soccer	67.7 KB
soccer_ayso_national_rules.txt	Soccer	29.8 KB
basketball_las_vegas_youth_rules.txt	Basketball	37.5 KB
volleyball_fort_collins_manual.txt	Volleyball	24.0 KB
cricket_uspl_t20_rules.txt	Cricket	61.4 KB
general_douglas_county_all_sports_rules.txt	Multi-sport	12.4 KB

Each file’s source URL, access date, byte size, and SHA-256 hash are recorded in `SOURCES.md` and `CORPUS_MANIFEST.json`.

Domain questions (committed before running any retrieval)

1. In AYSO soccer, how many players per side are allowed on the field for the 8U age division, and are goalkeepers used? (*expected source: soccer_ayso_national_rules.txt*)
2. In the City of Las Vegas youth basketball rules, at how many team fouls does the free-throw bonus (1-and-1) begin, and when does the double bonus begin? (*expected source: basketball_las_vegas_youth_rules.txt*)
3. In a USPL Twenty20 cricket match, how long is each session (innings), and how is that time calculated? (*expected source: cricket_uspl_t20_rules.txt*)
4. In the Fort Collins Adult Volleyball Manual, what is the time limit for a match, and how many games make up a match? (*expected source: volleyball_fort_collins_manual.txt*)
5. According to the Douglas County Parks & Recreation rules, what are the minimum and maximum roster sizes for adult volleyball? (*expected source: general_douglas_county_all_sports_rules.txt*)

Chunking pipelines

Each technique builds its own in-memory `VectorStoreIndex` (backed by `SimpleVectorStore`), using the shared `sentence-transformers/all-MiniLM-L6-v2` embedding model.

```
def build_token_index(documents, embed_model):
    splitter = TokenTextSplitter(
        chunk_size=256, chunk_overlap=20
    )
    nodes = splitter.get_nodes_from_documents(documents)
    return build_index(nodes, embed_model), nodes

def build_semantic_index(documents, embed_model):
    splitter = SemanticSplitterNodeParser.from_defaults(
        embed_model=embed_model, buffer_size=1
    )
    nodes = splitter.get_nodes_from_documents(documents)
    return build_index(nodes, embed_model), nodes

def build_sentence_window_index(documents, embed_model):
    splitter = SentenceWindowNodeParser.from_defaults(
        window_size=3
    )
    nodes = splitter.get_nodes_from_documents(documents)
    return build_index(nodes, embed_model), nodes
```

Retrieval-only function

Given a query and k , this computes the query embedding (dimension and first 8 values), retrieves the top- k nodes, and for each computes the store's own similarity score plus an independently-recomputed cosine similarity, chunk length, and preview:

```
def cosine_similarity(a, b):
    a = np.array(a); b = np.array(b)
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b)))

def retrieve_and_analyze(technique_name, index, embed_model, query, k=3):
    query_embedding = embed_model.get_query_embedding(query)
    query_vec = np.array(query_embedding)
    print(f"Query embedding dimension: {query_vec.shape[0]}")
    print(f"Query embedding first 8 values: {query_vec[:8]}")

    retriever = index.as_retriever(similarity_top_k=k)
    start = time.perf_counter()
    results = retriever.retrieve(query)
    latency_ms = (time.perf_counter() - start) * 1000

    doc_vecs, rows = [], []
    for rank, node_with_score in enumerate(results, start=1):
        chunk_text = node_with_score.node.get_content()
        store_score = node_with_score.score
        doc_embedding = embed_model.get_text_embedding(chunk_text)
        doc_vec = np.array(doc_embedding)
        doc_vecs.append(doc_vec)
        cos_sim = cosine_similarity(query_vec, doc_vec)
        rows.append({
            "rank": rank, "store_score": round(float(store_score), 4),
```

```

        "cosine_sim": round(cos_sim, 4), "chunk_len": len(chunk_text),
        "preview": chunk_text[:160].replace("\n", " "),
        "source_file": node_with_score.node.metadata.get("source_file"),
    })

doc_matrix = np.stack(doc_vecs) if doc_vecs else np.array([])
print(f"Query vector shape: {query_vec.shape}")
print(f"Stacked doc vectors shape: {doc_matrix.shape}")
return {"technique": technique_name, "query": query,
        "latency_ms": round(latency_ms, 2), "results": rows}

```

Real console output (excerpts)

Full raw output for all 15 (technique $\times$ question) combinations is saved in `reports/hw03/raw/chunking_comparison` and `.csv`. Representative excerpts, run against the real embedding model on the author's machine:

```
Loading embedding model (sentence-transformers/all-MiniLM-L6-v2)...
Model loaded.

Building indexes for all three techniques...
Building Token index...
  254 chunks produced.
Building Semantic index (this can take a while - embeds during chunking)...
  109 chunks produced.
Building Sentence-window index...
  2020 chunks produced.

=====
TECHNIQUE: Token QUERY: 'In the Fort Collins Adult Volleyball Manual, what
is the time limit for a match, and how many games make up a match?'
=====
Query embedding dimension: 384
Query embedding first 8 values: [ 0.0202 0.0205 -0.0108 -0.0633 -0.0011
 0.0348 -0.1015 -0.0374]
Query vector shape: (384,)
Stacked doc vectors shape: (3, 384)

rank store_score cosine_sim chunk_len preview
1 0.8049 0.7138 925 use basic formations for offense
                        and defense. Adult Volleyball...
2 0.7727 0.6186 966 get the nets set up. 2. Coin
                        Toss: Two options are available...
3 0.725 0.597 934 formats. If matches are running
                        late, the supervisor has...

=====
TECHNIQUE: SentenceWindow QUERY: (same query as above)
=====
rank store_score cosine_sim chunk_len preview
1 0.8558 0.6642 68 Time Limit: There will be a one
                        (1) hour time limit for the match.
2 0.8306 0.6614 57 Time Limit: Matches will be
                        scheduled every 40 minutes.
3 0.8037 0.6248 178 Scoring and Results of the Match:
                        Each match will consist of three
                        (3) games to twenty-five (25)...
```

Note the contrast: Token's top result (chunk_len 925) contains the answer somewhere inside a larger block of text, while Sentence-window's top result (chunk_len 68) is precisely the one sentence containing the answer – a clear illustration of the granularity difference between the two techniques.

1. Retrieval Quality

Technique	Chunks	Avg len	Top-1 cos	Mean@k cos	Recall@k	Latency (ms)
Token	254	952.8	0.7156	0.6684	5/5	18.51
Semantic	109	2055.2	0.6277	0.5928	4/5	14.69
Sentence-window	2020	110.9	0.6987	0.6339	5/5	55.73

A confidently-scored retrieval that does not contain the answer: For question 1 (AYSO 8U team size), Semantic chunking’s top-3 results all came from `soccer_ayso_basic_rules.txt` instead of the expected `soccer_ayso_national_rules.txt`, with a confident top cosine similarity of **0.7193**.

Why the embedding likely considered it similar: both documents are AYSO soccer rulebooks discussing overlapping topics (team size by age division, ”Law 3: The Number Of Players”) in very similar vocabulary. However, they are different editions – `soccer_ayso_basic_rules.txt` is the 2009-2010 booklet, while `soccer_ayso_national_rules.txt` is the 06/2018 national rules document – and they state *different numbers* for the same age group (the basic booklet says ”5-a-side for U-8”; the national rules table says ”8U: 4-a-side, no goalkeepers”). The embedding captures topical similarity (”this is about AYSO team sizes by age”) but cannot distinguish ”this is the current, precise answer” from ”this is an older, approximate statement on the same topic” – a genuine real-world corpus issue: two documents from the same organization, topically near-identical, reflecting different rule editions. A related pattern appeared across all three techniques for this same question: the national rules team-size table has many consecutive, near-identically-formatted rows (18U, 16U, 14U, 12U, 10U, 8U, 6U), and Token and Sentence-window sometimes retrieved *adjacent* rows (6U, or 18U/16U/14U) rather than precisely the 8U row.

2. Observations

Token chunking performed best on this corpus across every retrieval quality metric (highest top-1 cosine, highest mean@k cosine, perfect 5/5 recall) while also being fast. This corpus consists of plain regulatory/rulebook prose without strong natural ”topic shift” signals, so Semantic chunking’s boundary-detection had little to work with – it under-split the text into fewer, much larger chunks (avg 2,055 chars, with one outlier reaching 12,544 chars), which dilutes a specific fact inside a large block of loosely related text and lowers similarity scores. Sentence-window chunking, at the opposite extreme, achieved perfect recall and occasionally pinpoint-precise single-sentence matches (see the Fort Collins example above), but its very fine granularity (2,020 chunks averaging only 111 characters) meant searching over far more chunks at retrieval time, roughly tripling latency compared to Token (55.7ms vs. 18.5ms) without matching Token’s overall similarity scores.

The one recall failure (Semantic, described above) did not repeat across other queries – no other technique’s best-scored result missed its expected source file entirely, though near-miss row confusion (the 8U/6U table-row issue) appeared in multiple techniques for that same question, suggesting this specific query (asking for one numeric value buried in a repetitive table) was the hardest of the five regardless of chunking strategy.

3. Conclusion

For this corpus (community sports league rulebooks), **Token-based chunking is the best choice for deployment**. It achieved the highest retrieval quality on every measured metric (top-1 cosine, mean@k cosine, and perfect recall) while keeping latency low, roughly a third of Sentence-window’s latency for better similarity scores. Semantic chunking’s theoretical advantage (finding natural topic

boundaries) did not pay off here because this corpus’s plain, structurally repetitive regulatory prose lacks the strong topic shifts that technique is designed to detect, causing it to under-split and dilute specific facts inside overly large chunks. Sentence-window chunking is worth considering as a secondary technique specifically when pinpoint, single-fact precision matters more than latency, but its 3x latency cost is hard to justify here given Token already matches its recall while scoring higher on similarity.

AI_USE.md

1. What did you use an AI assistant for, and what did you do yourself?

I used Claude Code as a tutor while working on both parts of the assignment. It helped me understand new concepts before I implemented them, including:

How signed session cookies work.

Why a server-side revocation store is needed for logout and idle-timeout enforcement.

How embeddings and cosine similarity work, including working through a numerical example.

How the three chunking techniques differ.

I wrote the code myself, including:

`auth.py`

The Bootstrap templates

`build.corpus.py`

`rag.chunking_comparison.py`

`summarize_chunking_comparison.py`

I used Claude Code when I was unsure how to get started, became stuck on an error, or needed an explanation of a concept. I then ran and tested the code myself.

I also independently tested the complete login, logout, and idle-timeout flow in the browser and with curl. I verified that logged-out and expired session cookies could not be reused.

For Part 2, I ran the corpus download and RAG chunking pipeline on my own computer using real source documents and an actual embedding model. I also wrote the five domain questions and expected answers myself before running the retrieval code, as required by the assignment.

2. One AI-produced output that was wrong/unsuitable, or one thing you independently verified

The initial corpus setup used four source documents, one for each sport in my domain schema. Based on their estimated sizes, the corpus appeared to be large enough to meet the assignment's 200 KB requirement. However, this estimate turned out to be incorrect.

3. How did you detect the problem or verified the result?

I discovered the problem by running `build.corpus.py`. The script downloaded the source documents, extracted their text, calculated the total corpus size, and checked whether it met the 200 KB requirement.

The output showed:

Total corpus size: 190,657 bytes (186.2 KB) Meets 200KB requirement: False

This showed that the corpus was actually about 9.3 KB short of the requirement.

I would not have caught the issue by only looking at the document list or estimating the document sizes. Running the automated size check provided an objective verification of the actual corpus size.

4. What did you change, and why does it work now?

I added two more real, public, domain-relevant documents:

The fuller AYSO National Rules Regulations document.

A Douglas County Parks Recreation multi-sport policy document.

I then ran the same script again. The output confirmed that the corpus now met the requirement:

Total corpus size: 232,807 bytes (227.4 KB) Meets 200KB requirement: True

The fix works because I verified the result by running the automated size check again instead of assuming that adding more documents would be sufficient.

Additional Data-Quality Issue Discovered During Verification

A similar verification process revealed a data-quality issue in Part 2.

When I ran `find_semantic_miss.py` on the actual retrieval results, I found that Semantic chunking retrieved the wrong source document for one question, even though its cosine similarity was relatively high at 0.7193.

The retrieval returned an older 2009–2010 AYSO rules booklet instead of the expected 2018 AYSO National Rules document. Both documents discussed team sizes by age division, but they provided different numbers for the same age group.

I did not expect this issue when building the corpus. I found it only because I checked the retrieval results programmatically against the expected source file.

This demonstrates why I independently verified the outputs of the RAG pipeline rather than relying only on similarity scores. A relatively high cosine similarity does not necessarily mean that the retrieved document is the correct or most authoritative source.